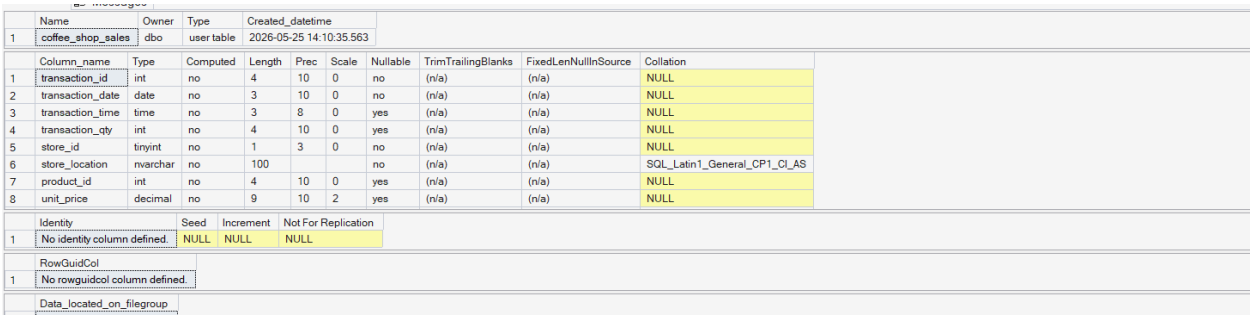
SQL Queries

create database coffee_shop_sales_db database created

I used SQL Server Management Studio's Import Flat File wizard. It automatically read the CSV file, inferred the schema, created a new table in the database, and loaded the data into it.

Checked the Meta Data of the table to check the data type detection:

```
exec sp_help 'dbo.coffee_shop_sales'
```



	Name	Owner	Type	Created_datetime
1	coffee_shop_sales	dbo	user table	2026-05-25 14:10:35.563

	Column_name	Type	Computed	Length	Prec	Scale	Nullable	TrimTrailingBlanks	FixedLenNullInSource	Collation
1	transaction_id	int	no	4	10	0	no	(n/a)	(n/a)	NULL
2	transaction_date	date	no	3	10	0	no	(n/a)	(n/a)	NULL
3	transaction_time	time	no	3	8	0	yes	(n/a)	(n/a)	NULL
4	transaction_qty	int	no	4	10	0	yes	(n/a)	(n/a)	NULL
5	store_id	tinyint	no	1	3	0	no	(n/a)	(n/a)	NULL
6	store_location	nvarchar	no	100			no	(n/a)	(n/a)	SQL_Latin1_General_CP1_CI_AS
7	product_id	int	no	4	10	0	yes	(n/a)	(n/a)	NULL
8	unit_price	decimal	no	9	10	2	yes	(n/a)	(n/a)	NULL

	Identity	Seed	Increment	Not For Replication
1	No identity column defined.	NULL	NULL	NULL

	RowGuidCol
1	No rowguidcol column defined.

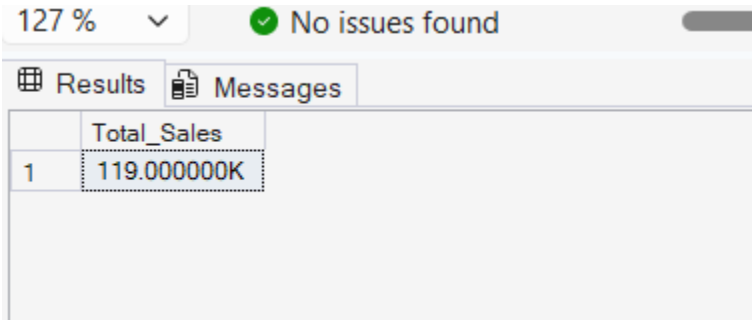
	Data_located_on_filegroup

Fixed the column data and data type.

```
-- update the date column to hh-mm-ss--
UPDATE dbo.coffee_shop_sales
SET transaction_time = CONVERT(TIME(0), transaction_time)
  -- change the data type fo the transaction_time column--
ALTER TABLE dbo.coffee_shop_sales
ALTER COLUMN transaction_time TIME(0)

--Change the data type of transaction_qty to INT from tiny int--
ALTER TABLE dbo.coffee_shop_sales
ALTER COLUMN transaction_qty INT

-- changed the data type of unit_price to decimal from float--
ALTER TABLE dbo.coffee_shop_sales
ALTER COLUMN unit_price Decimal (10,2)


-- changed the data type of product_id to decimal from float--
ALTER TABLE dbo.coffee_shop_sales
ALTER COLUMN product_id INT
```

1. Total Sales:

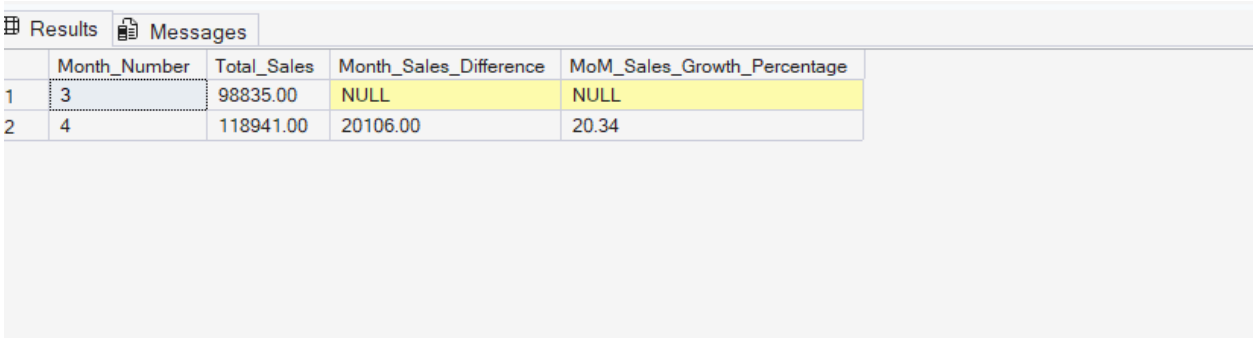
```
SELECT
   CONCAT(ROUND(SUM(unit_price * transaction_qty) / 1000, 0),'K') AS Total_Sales
FROM coffee_shop_sales
WHERE MONTH(transaction_date) = 4;
```



2 SQL query to find difference from the previous month and MOM growth in sales for the selected month:

```
WITH Monthly_Sales AS
(SELECT
      MONTH(transaction_date) AS Month_Number,
      ROUND(SUM(unit_price * transaction_qty), 0) AS Total_Sales
   FROM coffee_shop_sales
   WHERE MONTH(transaction_date) IN (3, 4)
   GROUP BY MONTH(transaction_date)
)

SELECT
   Month_Number,
   Total_Sales,
   -- Difference from Previous Month--
   ROUND(Total_Sales - LAG(Total_Sales, 1) OVER (ORDER BY Month_Number),2) AS
Month_Sales_Difference,
-- MoM Percentage Growth--
   ROUND((Total_Sales -LAG(Total_Sales, 1) OVER (ORDER BY Month_Number))/CAST(
        LAG(Total_Sales, 1) OVER (ORDER BY Month_Number)AS FLOAT) * 100,2) AS
MoM_Sales_Growth_Percentage
```

```
FROM Monthly_Sales
ORDER BY Month_Number;
```



Results | Messages

	Month_Number	Total_Sales	Month_Sales_Difference	MoM_Sales_Growth_Percentage
1	3	98835.00	NULL	NULL
2	4	118941.00	20106.00	20.34

3. Total Orders

```
SELECT
  COUNT(transaction_id) AS Total_Orders
FROM coffee_shop_sales
WHERE MONTH(transaction_date) = 4;   -- April
```

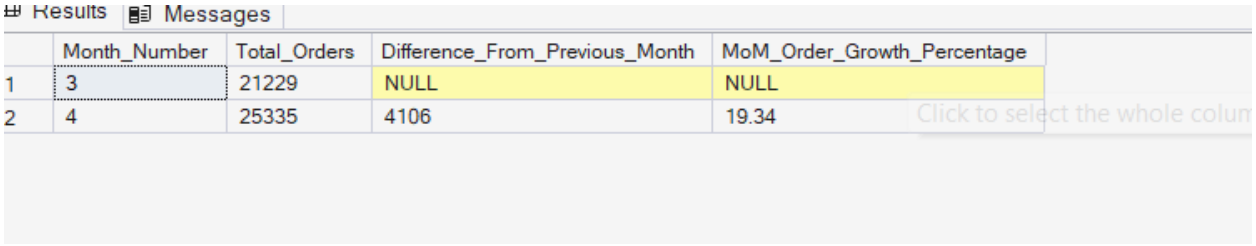
Results | Messages

	Total_Orders
1	25335

4. SQL query to find difference from the previous month and MOM growth in Orders for the selected month:

```
WITH Monthly_Orders AS
(SELECT
      MONTH(transaction_date) AS Month_Number,
      COUNT(transaction_id) AS Total_Orders
   FROM coffee_shop_sales
   WHERE MONTH(transaction_date) IN (3, 4)
   GROUP BY MONTH(transaction_date)
)

SELECT
   Month_Number,
   Total_Orders,

   -- Order Difference from Previous Month
```

```
   Total_Orders - LAG(Total_Orders, 1) OVER (ORDER BY Month_Number)
   AS Difference_From_Previous_Month,
-- MoM Order Growth Percentage--
   ROUND((Total_Orders -LAG(Total_Orders, 1) OVER (ORDER BY Month_Number))/
      CAST(LAG(Total_Orders, 1) OVER (ORDER BY Month_Number)AS FLOAT) * 100,2) AS
MoM_Order_Growth_Percentage
FROM Monthly_Orders;
```



Results | Messages

	Month_Number	Total_Orders	Difference_From_Previous_Month	MoM_Order_Growth_Percentage
1	3	21229	NULL	NULL
2	4	25335	4106	19.34

5. Total Orders:

```
SELECT
   SUM(transaction_qty) AS Total_Quantity_Sold
FROM coffee_shop_sales
WHERE MONTH(transaction_date) = 4;   -- April
```

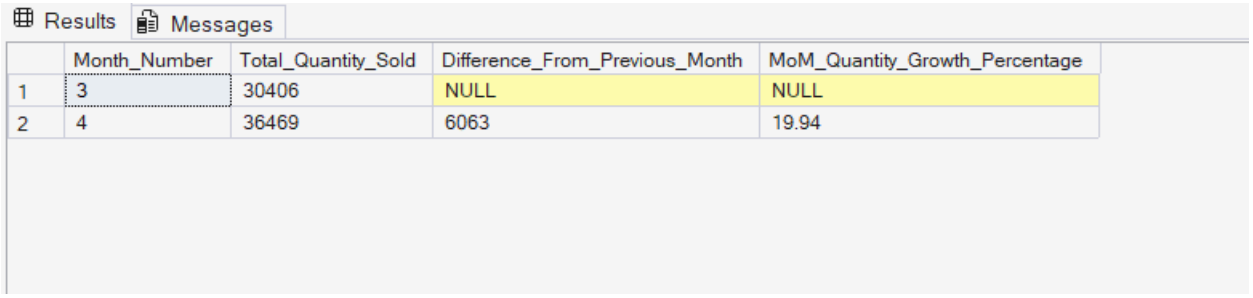
Results | Messages

	Total_Quantity_Sold
1	36469

6. SQL query to find difference from the previous month and MOM growth in Total Quanity Sold for the selected month:

```
WITH Monthly_Quantity_Sold AS
(SELECT
      MONTH(transaction_date) AS Month_Number,
      SUM(transaction_qty) AS Total_Quantity_Sold
   FROM coffee_shop_sales
   WHERE MONTH(transaction_date) IN (3, 4)
   GROUP BY MONTH(transaction_date)
```

```
)

SELECT
   Month_Number,
   Total_Quantity_Sold,

   -- Quantity Difference from Previous Month
   Total_Quantity_Sold - LAG(Total_Quantity_Sold, 1) OVER (ORDER BY Month_Number) AS
Difference_From_Previous_Month,

   -- MoM Quantity Growth Percentage
   ROUND((Total_Quantity_Sold - LAG(Total_Quantity_Sold, 1)OVER (ORDER BY Month_Number))/
      CAST(LAG(Total_Quantity_Sold, 1)OVER (ORDER BY Month_Number)AS FLOAT) * 100,2) AS
MoM_Quantity_Growth_Percentage

FROM Monthly_Quantity_Sold;
```



Results Messages

	Month_Number	Total_Quantity_Sold	Difference_From_Previous_Month	MoM_Quantity_Growth_Percentage
1	3	30406	NULL	NULL
2	4	36469	6063	19.94

7. Return KPI Values for a specific date for Calendar Heat Map's Tooltip

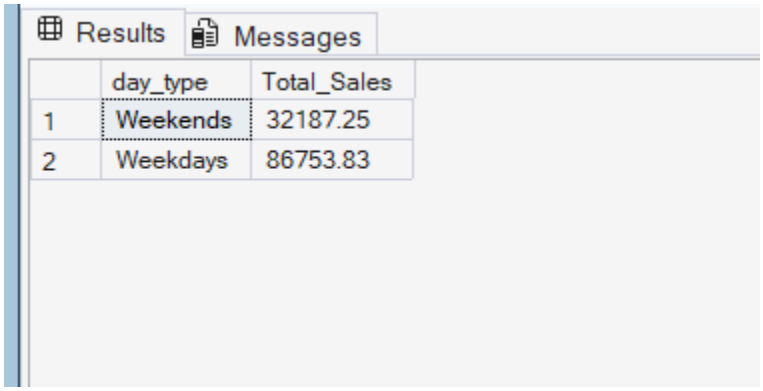
```
SELECT
   SUM(unit_price * transaction_qty) AS Total_Sales,
   SUM(transaction_qty)          AS Total_Qty_Sold,
   COUNT(transaction_id)         AS Total_Orders
FROM coffee_shop_sales
WHERE transaction_date = '2025-04-18';
```

Results Messages

	Total_Sales	Total_Qty_Sold	Total_Orders
	4354.07	1275	910

8. Total Sales Analysis for Weekdays and Weekends;

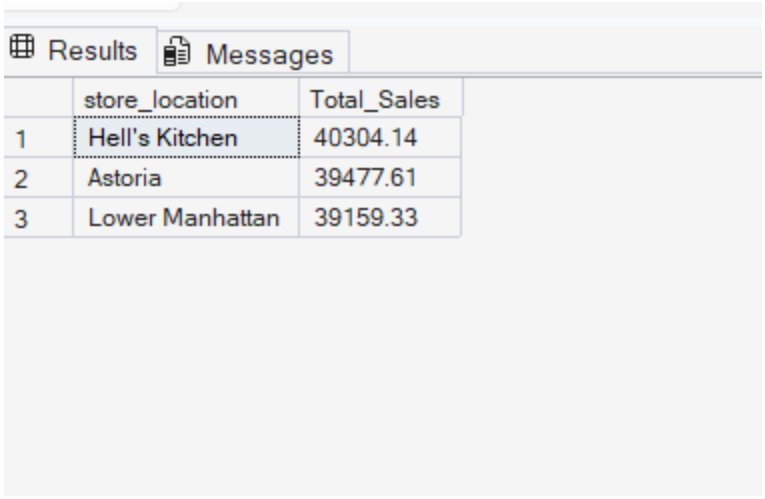
```
SELECT
   CASE WHEN DATEPART(WEEKDAY, transaction_date) IN (1, 7) THEN 'Weekends'
   ELSE 'Weekdays'
   END AS day_type,
SUM(unit_price * transaction_qty) AS Total_Sales
FROM coffee_shop_sales
WHERE MONTH(transaction_date) = 4   -- April
GROUP BY
CASE WHEN DATEPART(WEEKDAY, transaction_date) IN (1, 7) THEN 'Weekends'
  ELSE 'Weekdays'
   END;
```



	day_type	Total_Sales
1	Weekends	32187.25
2	Weekdays	86753.83

9. Total Sales Analysis by store location:

```
SELECT
   store_location,
   SUM(unit_price * transaction_qty) AS Total_Sales
FROM coffee_shop_sales
WHERE MONTH(transaction_date) = 4  -- April
GROUP BY store_location
ORDER BY Total_Sales DESC;
```



Results | Messages

	store_location	Total_Sales
1	Hell's Kitchen	40304.14
2	Astoria	39477.61
3	Lower Manhattan	39159.33

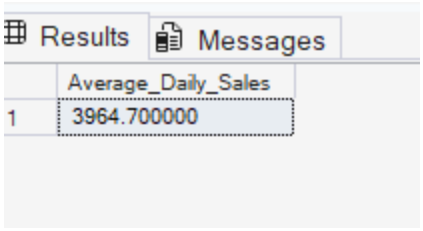
10. Daily sales calculation for the selected month:

```
SELECT
   transaction_date,
   SUM(unit_price * transaction_qty) AS Total_Sales
FROM coffee_shop_sales
WHERE MONTH(transaction_date) = 4
GROUP BY transaction_date
ORDER BY transaction_date;
```

	transaction_date	Total_Sales
1	2025-04-01	3699.90
2	2025-04-02	3575.85
3	2025-04-03	3604.95
4	2025-04-04	3327.30
5	2025-04-05	3552.70
6	2025-04-06	3250.20
7	2025-04-07	3682.80
8	2025-04-08	4573.06
9	2025-04-09	4088.88
10	2025-04-10	4220.30
11	2025-04-11	3852.86
12	2025-04-12	4040.18
13	2025-04-13	4114.53
14	2025-04-14	4131.25
15	2025-04-15	4121.79
16	2025-04-16	4500.39
17	2025-04-17	4332.10
18	2025-04-18	4354.07
19	2025-04-19	4318.31
20	2025-04-20	3924.78
21	2025-04-21	4005.38
22	2025-04-22	3961.58
23	2025-04-23	4321.29
24	2025-04-24	4265.45
25	2025-04-25	4255.00
26	2025-04-26	4559.45
27	2025-04-27	4427.10
28	2025-04-28	3373.80
29	2025-04-29	2953.50
30	2025-04-30	3552.33

11. Average Daily Sales:

```
SELECT
   ROUND(AVG(Daily_Sales), 2) AS Average_Daily_Sales
FROM (
   SELECT
      transaction_date,
      SUM(unit_price * transaction_qty) AS Daily_Sales
   FROM coffee_shop_sales
   WHERE MONTH(transaction_date) = 4
   GROUP BY transaction_date
) AS DailyTable;
```



12. Compare daily sales with overall average:

```
WITH DailySales AS (
   SELECT
      CAST(transaction_date AS DATE) AS day_of_month,
      SUM(unit_price * transaction_qty) AS total_sales,
      AVG(SUM(unit_price * transaction_qty)) OVER () AS avg_sales
   FROM coffee_shop_sales
   WHERE MONTH(transaction_date) = 4
   GROUP BY CAST(transaction_date AS DATE)
)

SELECT
   day_of_month,
   total_sales,
   avg_sales,
   CASE
      WHEN total_sales > avg_sales THEN 'Above Average'
      WHEN total_sales < avg_sales THEN 'Below Average'
      ELSE 'Equal to Average'
   END AS sales_status
FROM DailySales
ORDER BY day_of_month;
```

day_of_month	total_sales	avg_sales	sales_status
2025-04-01	3699.90	3964.702666	Below Average
2025-04-02	3575.85	3964.702666	Below Average
2025-04-03	3604.95	3964.702666	Below Average
2025-04-04	3327.30	3964.702666	Below Average
2025-04-05	3552.70	3964.702666	Below Average
2025-04-06	3250.20	3964.702666	Below Average
2025-04-07	3682.80	3964.702666	Below Average
2025-04-08	4573.06	3964.702666	Above Average
2025-04-09	4088.88	3964.702666	Above Average
2025-04-10	4220.30	3964.702666	Above Average
2025-04-11	3852.86	3964.702666	Below Average
2025-04-12	4040.18	3964.702666	Above Average
2025-04-13	4114.53	3964.702666	Above Average

13. Sales by Product Category:

```
SELECT
```

```
    product_category,
    SUM(unit_price * transaction_qty) AS Total_Sales
FROM coffee_shop_sales
WHERE MONTH(transaction_date) = 4
GROUP BY product_category
ORDER BY Total_Sales DESC;
```

	product_category	Total_Sales
1	Coffee	45971.20
2	Tea	33356.95
3	Bakery	14021.70
4	Drinking Chocolate	12266.75
5	Coffee beans	6824.70
6	Branded	2379.00
7	Loose Tea	1829.15
8	Flavours	1418.40
9	Packaged Chocolate	873.23

14. Top 10 Product Type by Total Revenue:

```
SELECT TOP 10
    product_type,
    SUM(unit_price * transaction_qty) AS Total_Sales
FROM coffee_shop_sales
WHERE MONTH(transaction_date) = 4
  AND product_category = 'Coffee'
GROUP BY product_type
ORDER BY Total_Sales DESC;
```

	product_type	Total_Sales
1	Barista Espresso	15555.90
2	Gourmet brewed coffee	11820.50
3	Organic brewed coffee	6611.00
4	Premium brewed coffee	6568.30
5	Drip coffee	5415.50

15. Calculation for Heat Map Tooltip(hours and day context)

```
SELECT
   SUM(unit_price * transaction_qty) AS Total_Sales,
   SUM(transaction_qty)          AS Total_Qty_Sold,
   COUNT(*)                      AS Total_Orders
FROM coffee_shop_sales
WHERE MONTH(transaction_date) = 4
 AND DATEPART(WEEKDAY, transaction_date) = 2  -- Monday
 AND DATEPART(HOUR, transaction_time) = 8;
```

	Total_Sales	Total_Qty_Sold	Total_Orders
1	2078.03	632	459

16. Total Sales for peak business hours

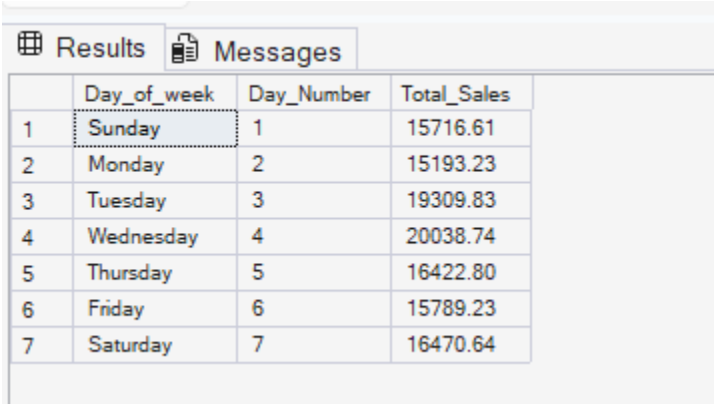
```
SELECT
   DATEPART(HOUR, transaction_time) AS hour_of_day,
   SUM(unit_price * transaction_qty) AS Total_Sales
FROM coffee_shop_sales
WHERE MONTH(transaction_date) = 5
GROUP BY DATEPART(HOUR, transaction_time)
ORDER BY hour_of_day;
```

	hour_of_day	Total_Sales
1	6	3772.28
2	7	10500.67
3	8	13723.07
4	9	14609.25
5	10	15450.93
6	11	8216.87
7	12	6902.49
8	13	6553.33
9	14	6933.05
10	15	7144.65
11	16	7065.31
12	17	7012.89

17. Helps to find the total sales for each day of the week for the selected month

```
SELECT
   DATENAME(WEEKDAY, transaction_date) AS Day_of_week,
   DATEPART(WEEKDAY, transaction_date) AS Day_Number,
   SUM(unit_price * transaction_qty) AS Total_Sales
FROM coffee_shop_sales
WHERE MONTH(transaction_date) = 4
GROUP BY
```

```
    DATENAME(WEEKDAY, transaction_date),
    DATEPART(WEEKDAY, transaction_date)
ORDER BY Day_Number;
```



Results | Messages

	Day_of_week	Day_Number	Total_Sales
1	Sunday	1	15716.61
2	Monday	2	15193.23
3	Tuesday	3	19309.83
4	Wednesday	4	20038.74
5	Thursday	5	16422.80
6	Friday	6	15789.23
7	Saturday	7	16470.64